

Neural networks and SVM

Câu 1: Perceptron và SVM

a) Huấn luyện Perceptron

Cho tập dữ liệu và các tham số khởi tạo:

- $w_0 = -0.5, w_1 = 0.4, w_2 = 0.5$ (với $x_0 = 1$)
- Tốc độ học $\eta = 0.2$
- Hàm kích hoạt: $y = 1$ nếu $\sum_{i=0}^2 x_i w_i > 0$, ngược lại $y = -1$.

Epoch 1:

1. **Điểm 1:** $\mathbf{x}^{(1)} = (0, 0)$, nhãn $t = -1$.

$$\text{net} = 1(-0.5) + 0(0.4) + 0(0.5) = -0.5 \leq 0 \implies \hat{y} = -1 \quad (\text{Đúng})$$

Không cập nhật trọng số.

2. **Điểm 2:** $\mathbf{x}^{(2)} = (0, 1)$, nhãn $t = 1$.

$$\text{net} = 1(-0.5) + 0(0.4) + 1(0.5) = 0 \leq 0 \implies \hat{y} = -1 \quad (\text{Sai})$$

Cập nhật trọng số:

$$w_0 = -0.5 + 0.2(1 - (-1)) \times 1 = -0.1$$

$$w_1 = 0.4 + 0.2(1 - (-1)) \times 0 = 0.4$$

$$w_2 = 0.5 + 0.2(1 - (-1)) \times 1 = 0.9$$

Bộ trọng số mới: $\mathbf{w} = [-0.1, 0.4, 0.9]$.

3. **Điểm 3:** $\mathbf{x}^{(3)} = (1, 0)$, nhãn $t = 1$.

$$\text{net} = 1(-0.1) + 1(0.4) + 0(0.9) = 0.3 > 0 \implies \hat{y} = 1 \quad (\text{Đúng})$$

Không cập nhật trọng số.

4. **Điểm 4:** $\mathbf{x}^{(4)} = (1, 1)$, nhãn $t = 1$.

$$\text{net} = 1(-0.1) + 1(0.4) + 1(0.9) = 1.2 > 0 \implies \hat{y} = 1 \quad (\text{Đúng})$$

Không cập nhật trọng số.

Epoch 2:

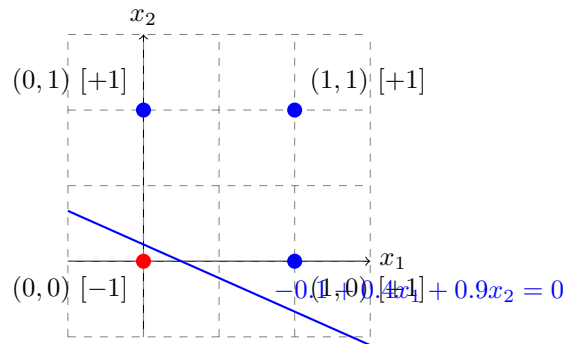
Kiểm tra lại toàn bộ tập dữ liệu với bộ trọng số $\mathbf{w} = [-0.1, 0.4, 0.9]$:

- Với $(0, 0)$: $\text{net} = -0.1 \leq 0 \implies \hat{y} = -1$ (Đúng).
- Với $(0, 1)$: $\text{net} = 0.8 > 0 \implies \hat{y} = 1$ (Đúng).
- Với $(1, 0)$: $\text{net} = 0.3 > 0 \implies \hat{y} = 1$ (Đúng).
- Với $(1, 1)$: $\text{net} = 1.2 > 0 \implies \hat{y} = 1$ (Đúng).

Thuật toán hội tụ. Phương trình đường biên tìm được:

$$-0.1 + 0.4x_1 + 0.9x_2 = 0 \iff x_2 = -0.44x_1 + 0.11$$

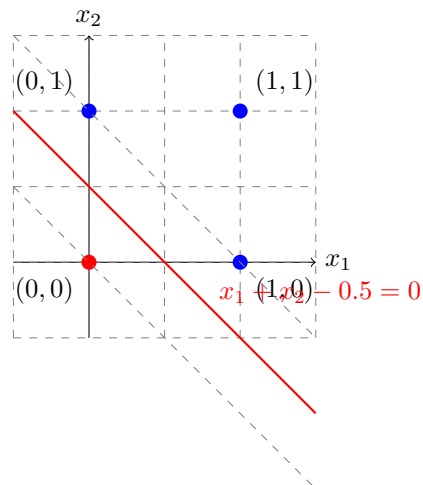
b) Đồ thị mô hình Perceptron



c) Đồ thị mô hình SVM lý tưởng

Ranh giới phân lớp tối ưu của SVM tối đa hóa khoảng cách (margin) giữa hai lớp.

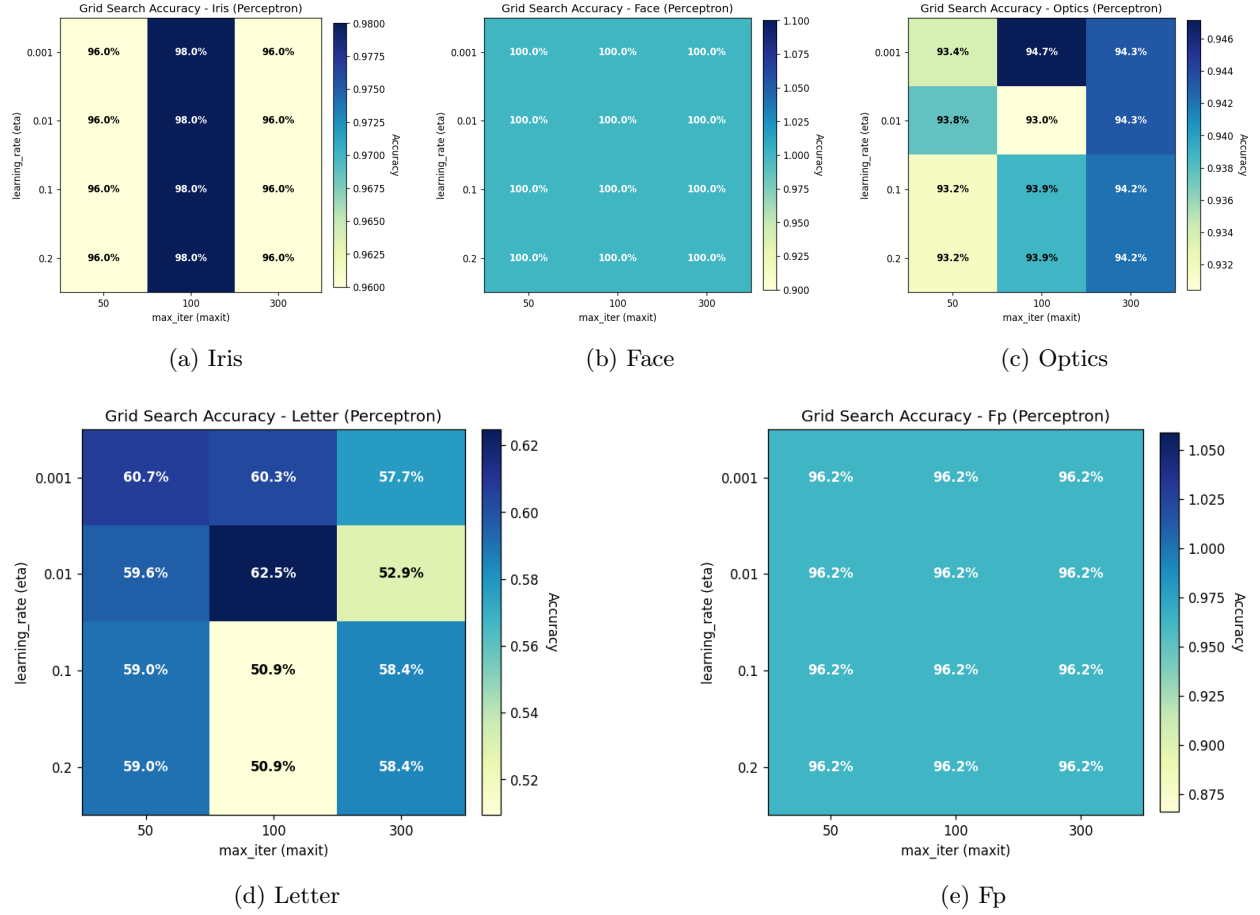
- Đường biên tối ưu: $x_1 + x_2 - 0.5 = 0$.
- Các vector hỗ trợ (Support Vectors) nằm trên hai đường biên lề: $x_1 + x_2 = 0$ và $x_1 + x_2 = 1$.



Câu 2: Cài đặt Perceptron và nhận xét

a) Quá trình tối ưu hóa tham số (Grid Search)

Để tìm ra cấu hình hoạt động tốt nhất cho thuật toán Perceptron tự triển khai, một quá trình tìm kiếm lưới (Grid Search) đã được thực hiện bằng cách thay đổi giá trị tốc độ học $\eta \in \{0.001, 0.01, 0.1, 0.2\}$ và số vòng lặp tối đa $maxit \in \{50, 100, 300\}$.



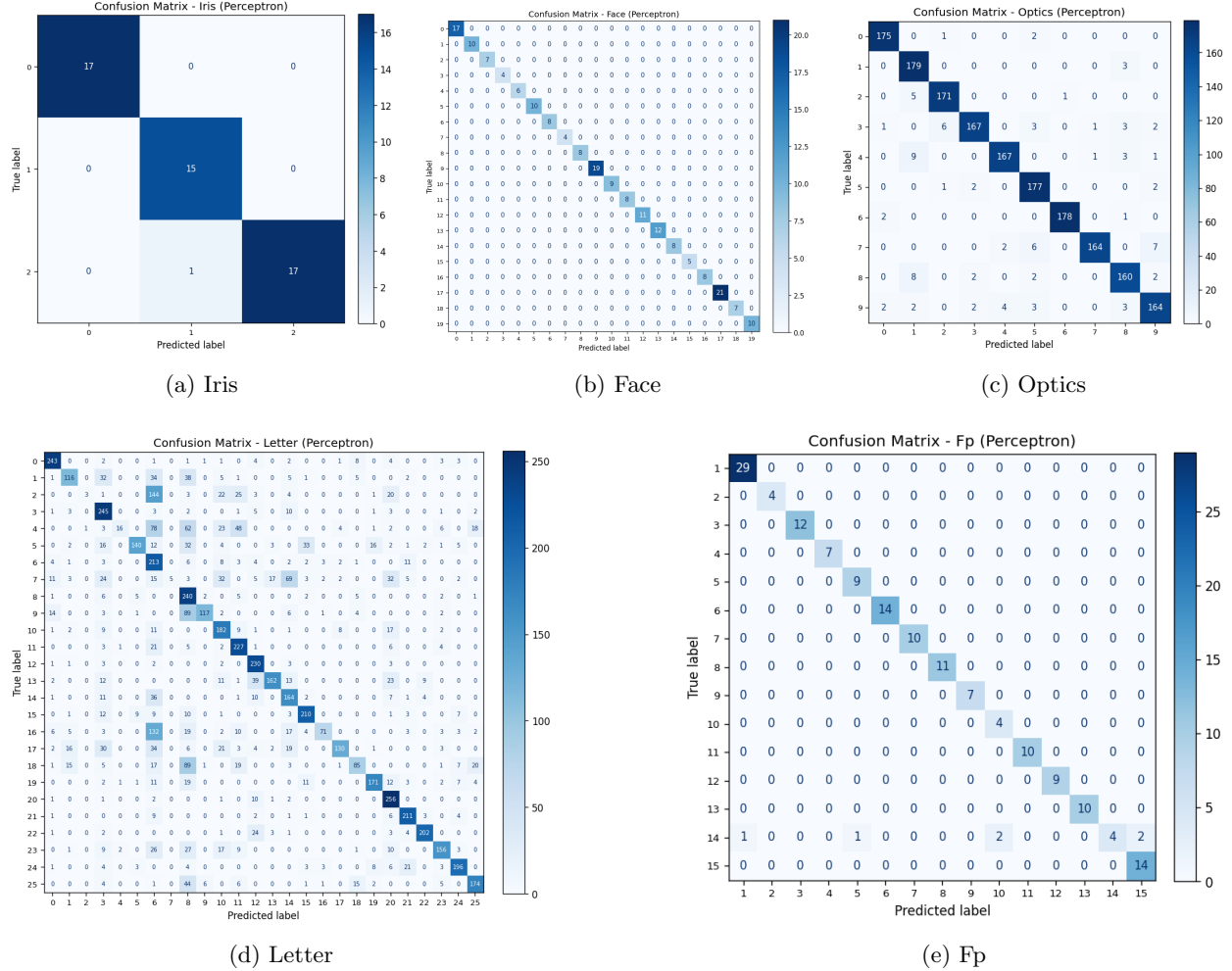
Hình 1: Bản đồ nhiệt thể hiện độ chính xác của Perceptron theo các bộ tham số khác nhau

Dựa vào kết quả tìm kiếm trên lưới tham số (Hình 1), chúng tôi rút ra một số nhận xét sau về độ nhạy của thuật toán:

- **Ít chịu ảnh hưởng của tham số (Face, Fp):** Đối với các tập dữ liệu có ranh giới phân biệt tuyến tính cao như *Face* (đạt 100.0%) hay *Fp* (đạt 96.2%), độ chính xác giữ nguyên không đổi trên toàn bộ các tổ hợp kiểm thử. Điều này chứng tỏ mô hình đạt điểm hội tụ rất nhanh (ngay dưới 50 vòng lặp đầu tiên).
- **Phụ thuộc vào số vòng lặp (Iris):** Tập *Iris* đạt độ chính xác cao nhất ở mức 98.0% duy nhất tại giá trị $maxit = 100$. Khi chạy ít hơn ($maxit = 50$) hoặc chạy quá nhiều ($maxit = 300$), mô hình có xu hướng chưa hội tụ đủ hoặc bị dao động quá đà quanh ranh giới tối ưu, làm giảm nhẹ hiệu năng xuống còn 96.0%.
- **Phụ thuộc chặt chẽ vào cả hai tham số (Letter, Optics):** Tập *Optics* đạt đỉnh 94.7% ở cấu hình $\eta = 0.001$ và $maxit = 100$. Đặc biệt, tập phức tạp như *Letter* có mức độ dao động rất lớn: độ chính xác cao nhất đạt 62.5% tại $\eta = 0.01$, $maxit = 100$ nhưng có thể tụt sâu xuống mức 50.9% khi tăng tốc độ học hoặc tăng số lượt lặp lên quá cao, gây ra hiện tượng lệch pha hoặc mất tính ổn định khi cập nhật trọng số liên tục.

b) Kết quả phân lớp chi tiết với bộ tham số tối ưu

Sử dụng bộ tham số tốt nhất được tìm thấy từ quá trình Grid Search để tiến hành huấn luyện chính thức mô hình Perceptron, ta thu được kết quả phân lớp biểu diễn qua các ma trận nhầm lẫn sau:



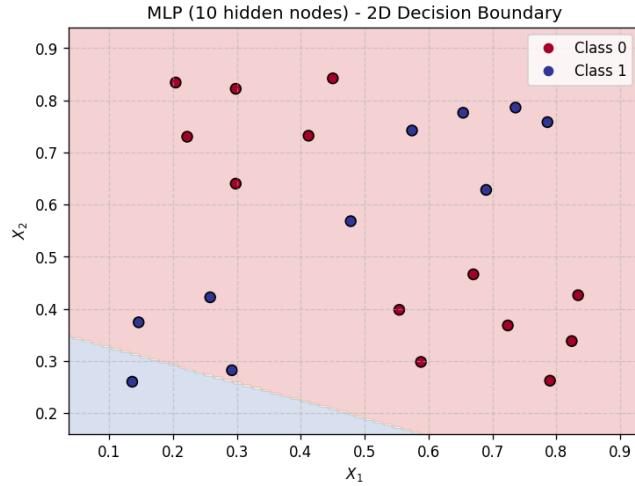
Hình 2: Ma trận nhầm lẫn của mô hình Perceptron đã tối ưu hóa tham số

Phân tích chi tiết ma trận nhầm lẫn:

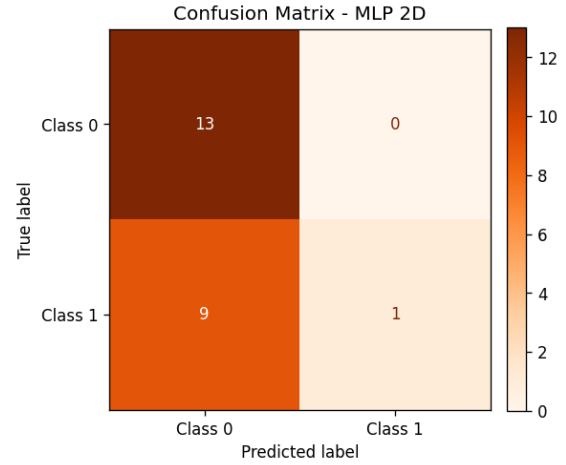
- **Tập Iris và Face:** Mô hình hoạt động ổn định. Tập *Face* phân loại đúng hoàn toàn không sai sót, tập *Iris* chỉ nhầm lẫn duy nhất 1 điểm dữ liệu của lớp 2 sang lớp 1.
- **Tập Optics và Fp:** Kết quả phân loại tương đối tốt. Với *Optics*, các chữ số viết tay đa phần được nhận diện chính xác, chỉ có một số lỗi nhỏ phân bố rải rác ngoài đường chéo chính (do sự tương đồng hình học tự nhiên giữa các số như 3 và 8). Trên tập *Fp*, ngoại trừ các lớp cuối (như lớp 14 bị nhầm lẫn rải rác sang lớp 1, 5, 10, 15 do tính chất đặc trưng bị trùng lặp nhẹ), các lớp còn lại đều được phân tách rõ rệt.
- **Tập Letter:** Ma trận nhầm lẫn bị mờ nhòe với sai số trải rộng khắp nơi. Do cấu trúc ký tự đa lớp (26 lớp) mang tính chất phi tuyến cực kỳ phức tạp và có tính chồng chéo hình học rất cao (ví dụ: chữ O dễ nhầm với Q, P dễ nhầm với R), bộ phân lớp tuyến tính như Perceptron không thể xây dựng được các mặt phẳng phân chia hiệu quả trong không gian này.

Câu 3: Cài đặt và đánh giá mô hình Perceptron

Mô hình MLP được huấn luyện trên tập dữ liệu gồm 23 điểm đặc trưng trong không gian 2D. Kết quả biểu diễn đường biên quyết định và ma trận nhầm lẫn tương ứng được thể hiện dưới đây:



(a) Đường biên phân lớp 2D của MLP



(b) Ma trận nhầm lẫn tương ứng

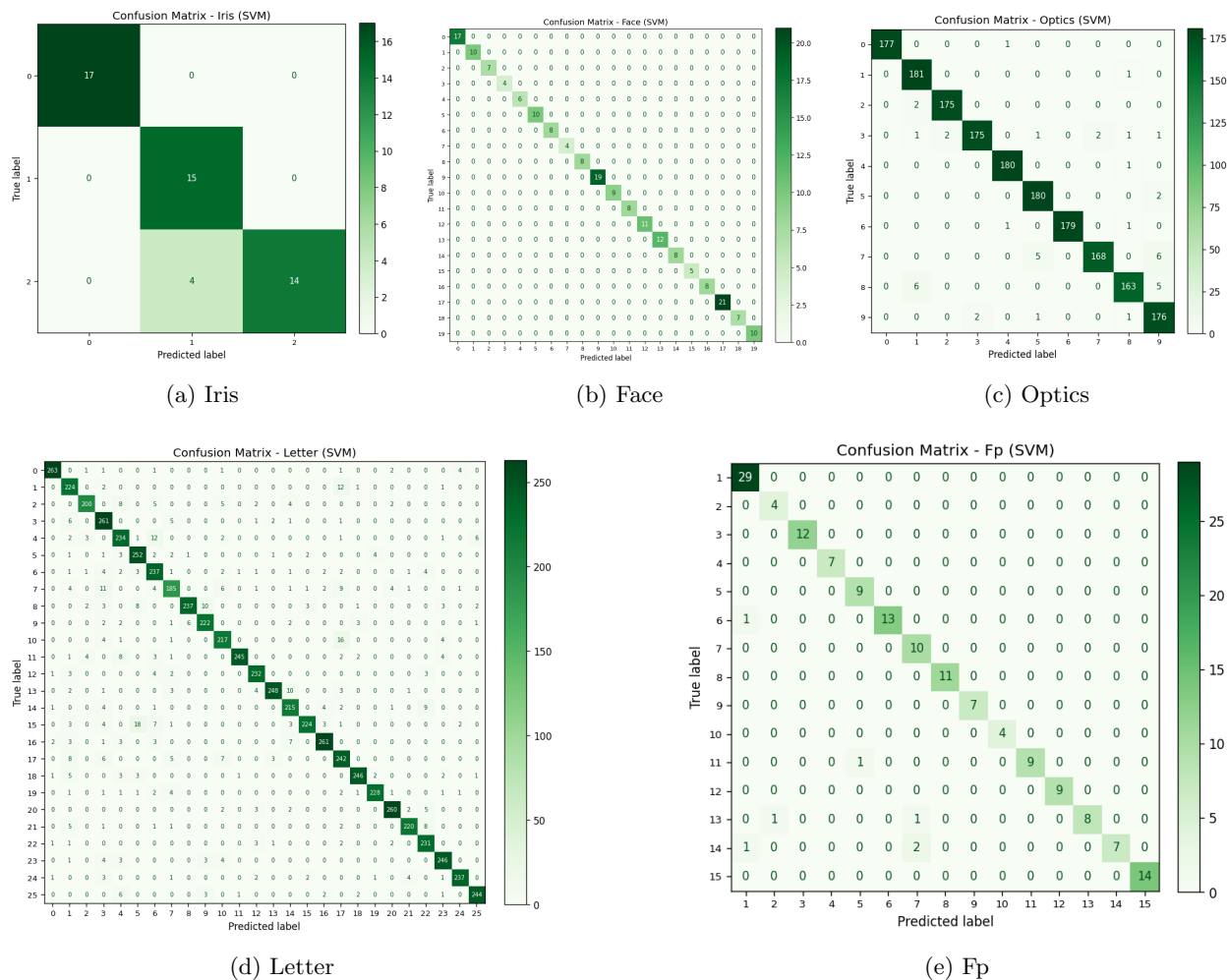
Hình 3: Kết quả thực nghiệm phân lớp dữ liệu 2D bằng mạng MLP

Nhận xét và phân tích lỗi: Dựa vào đồ thị quyết định (Hình 3a) và ma trận nhầm lẫn (Hình 3b), mô hình MLP với cấu trúc hiện tại chưa đạt được hiệu năng tối ưu trên tập dữ liệu này:

- **Hiện tượng Underfitting:** Đường biên quyết định phân lớp thu được gần như là một đường thẳng cắt ngang góc dưới bên trái của hệ trục tọa độ, chưa thể hiện được tính phi tuyến cần có để bao quanh nhóm dữ liệu.
- **Lệch phân lớp:** Do đường biên bị lệch, có tới 9/10 mẫu thuộc Class 1 bị mô hình dự đoán sai sang Class 0 (độ chính xác của Class 1 chỉ đạt 10%). Mẫu dữ liệu Class 0 được phân lớp đúng hoàn toàn (13/13 mẫu).
- **Nguyên nhân và giải pháp đề xuất:** Hiện tượng này thường xảy ra khi quá trình tối ưu hóa mạng (Adam optimizer) bị kẹt ở điểm cực tiểu cục bộ (local minima), hoặc số lượng vòng lặp huấn luyện chưa đủ lớn để mạng cập nhật trọng số hiệu quả. Để cải thiện, cần thử nghiệm thay đổi giá trị khởi tạo trọng số ngẫu nhiên khác (*random_state*), tăng nhẹ số node ẩn, hoặc áp dụng kỹ thuật chuẩn hóa dữ liệu đầu vào trước khi huấn luyện.

Câu 4: Cài đặt phân lớp bằng SVM

Mô hình SVM sử dụng hàm nhân RBF (Kernel Trick) cho thấy sức mạnh vượt trội khi giải quyết các bài toán phân lớp phi tuyến phức tạp:



Hình 4: Ma trận nhầm lẫn của mô hình SVM sử dụng nhân RBF

So sánh đối chiếu hiệu năng giữa Perceptron và SVM

Để có cái nhìn toàn diện về ưu và nhược điểm của hai phương pháp tiếp cận tuyến tính (Perceptron) và phi tuyến (SVM RBF), chúng tôi tổng hợp độ chính xác kiểm thử trên các tập dữ liệu dưới bảng sau:

Bảng 1: So sánh độ chính xác (%) giữa Perceptron (Đã tối ưu) và SVM (RBF)

Tập dữ liệu	Perceptron Scratch (%)	SVM RBF (%)
Iris	98.00%	92.00%
Face	100.00%	100.00%
Optics	94.70%	97.50%
Letter	62.50%	91.67%
Fp	96.20%	94.30%

Nhận xét và thảo luận:

- **Hiệu năng đột phá trên tập dữ liệu phi tuyến phức tạp:** Sự khác biệt rõ ràng nhất xuất hiện ở tập dữ liệu *Letter*. Sử dụng nhân RBF biến đổi không gian giúp SVM đạt độ chính xác nhảy vọt lên 91.67% so với mức trần nghèo nàn 62.50% của Perceptron. Biểu đồ ma trận nhầm lẫn (Hình 4d) cho thấy cấu trúc chéo chính cực kỳ sắc nét, giải quyết triệt để sự bối rối phân lớp của mô hình tuyến tính cũ. Tập *Optics* cũng chứng kiến sự cải thiện tương đương (tăng từ 94.70% lên 97.50%).
- **Hiệu ứng Overfitting nhẹ của SVM trên tập dữ liệu nhỏ:** Đáng chú ý, đối với tập dữ liệu đơn giản và có kích thước siêu nhỏ như *Iris*, Perceptron tuyến tính lại cho kết quả kiểm thử tốt hơn so với SVM RBF (98.00% so với 92.00%, tương ứng phân loại sai 1 mẫu so với 4 mẫu). Điều này phản ánh đặc tính thực tế: khi dữ liệu quá ít và cấu trúc phân chia đơn giản, việc áp dụng các hàm phi tuyến mạnh mẽ đôi khi lại có xu hướng làm cong ranh giới quyết định quá mức, dẫn đến khả năng tổng quát hóa trên tập kiểm tra kém hơn một bộ phân lớp tuyến tính thuần túy.
- **Đạt độ chính xác tối đa tuyệt đối:** Trên tập *Face*, cả hai phương pháp đều hoàn thành tốt nhiệm vụ phân loại chính xác hoàn toàn (100.00%), cho thấy tập dữ liệu này sở hữu các thuộc tính đặc trưng cực kỳ phân biệt và có tính tách biệt cao trong cả không gian tuyến tính lẫn phi tuyến.